

Node.js HTTP Module — Key Points

The Native JavaScript Gateway Contract

Source: <https://nodejs.org/api/http.html>

The Native Callback Contract

In Node.js, the gateway contract between the runtime and the application is not WSGI or ASGI. Instead, it is the native '(req, res)' listener passed to `http.createServer()`. The HTTP module handles TCP socket stream management, HTTP start-line and header parsing, and chunk accumulation, surfacing these as JavaScript objects.

`http.createServer([requestListener])`

Returns a new instance of `http.Server`. The `requestListener` is automatically bound to the 'request' event of the server. Every incoming HTTP connection triggers this callback once the headers have been parsed.

`http.IncomingMessage (req)`

- Implements the Readable Stream interface.
- `req.method`: The HTTP method as a string (e.g. 'GET').
- `req.url`: The raw request path and query string (e.g. '/bookmarks?skip=5').
- `req.headers`: The parsed headers object. Node.js automatically downcases all header names to guarantee case-insensitive lookups.
- Body reading: Because 'req' is a stream, the request body is not buffered in memory by default. You must listen to `req.on('data', chunk => ...)` to accumulate the body, and `req.on('end', ...)` to finish.

`http.ServerResponse (res)`

- Implements the Writable Stream interface.
- `res.writeHead(statusCode, [statusMessage], [headers])`: Sends the HTTP status line and response headers to the client. This replaces manually assembling 'HTTP/1.1 200 OK\r\n'.
- `res.write(chunk)`: Streams a chunk of the response body to the client TCP socket.
- `res.end([data])`: Signals to the server that all headers and body have been sent. This flushes the stream and terminates the HTTP response.